

11. Adding a New node at the beginning of list

Definition:-

Adding a node at the beginning of a Singly linked list means inserting a new node before the first node. After insertion, the new node becomes the head of the linked list.

Algorithm:-

1. Start.
2. Create a new node using dynamic memory allocation
3. Check wheather memory is allocated successfully
4. Enter the data into the new node
5. Set the next pointer of the new node to the current head.
6. Update the head pointer to point to the new node
7. The new node becomes the first node of the linked list
8. Stop.

C Program:-

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *head = NULL;

void insertBeginning()
{
    struct node *newnode;
    newnode = (struct node *)
    malloc (size of (struct node));
```

```

printf("Enter data: ");
scanf("%d", &newnode->data);
newnode->next = head;
head = newnode;
printf("Node inserted successfully.\n");
}

```

Time Complexity:-

- $O(1)$ [constant time]

Space Complexity:-

- $O(1)$ [only one extra node is created]

This is a complete 5-mark exam answer that is suitable for waiting in you.

37 Finding the length of a Singly Linked List

Definition:-

Finding the length of a linked list means counting the number of nodes present in the list by traversing it from first node to the last node.

Algorithm:-

1. Start
2. Initialize a pointer temp to the head node
3. Initialize a variable count = 0.
4. Traverse the linked list until temp becomes NULL.
5. Increment count for each node visited

Lower of tower.

Code:-

```
void TdH (int n, char s, char A, char D)
{
    if (n == 1)
    {
        printf ("Move disk from %c to %c \n", s, D);
        return;
    }
    TdH (n-1, s, D, A);
    printf ("Move disk %c from %c to %c \n", n, s, D);
    TdH (n-1, A, s, A);
}
```

Eq:-

Initial State:-

Source (A)	Aux (B)	Disk (C)
1		
2		
3		

Step 1:- Move disk from A $\rightarrow$ B

Source (A)	Aux (B)	Disk (C)
2		1
3		

Step 2:- Move Disk from A $\rightarrow$ B

Source (A)	Aux (B)	Disk (C)
3	2	1

Step 3:- Move disk 1 from C → B

Source(A)	Aux(B)	Disk(C)
3	1	
	2	

Step 4:- Move Disk 3 from A → C

Source(A)	Aux(B)	Disk(C)
	1	3
	2	

Step 5:- Move disk 1 from B → A

Source(A)	Aux(B)	Disk(C)
1	2	2

Step 6:- Move disk 2 from B → C

Source(A)	Aux(B)	Disk(C)
1		2
		2

Step 7:- Move disk 1 from A → C

Source(A)	Aux(B)	Disk(C)
		1
		2
		3

* Display & Print the Data:-

void display()

✓

struct node * temp = root;

while (temp != NULL)

{

Print ("y.d →", temp → data);

temp = temp → right;

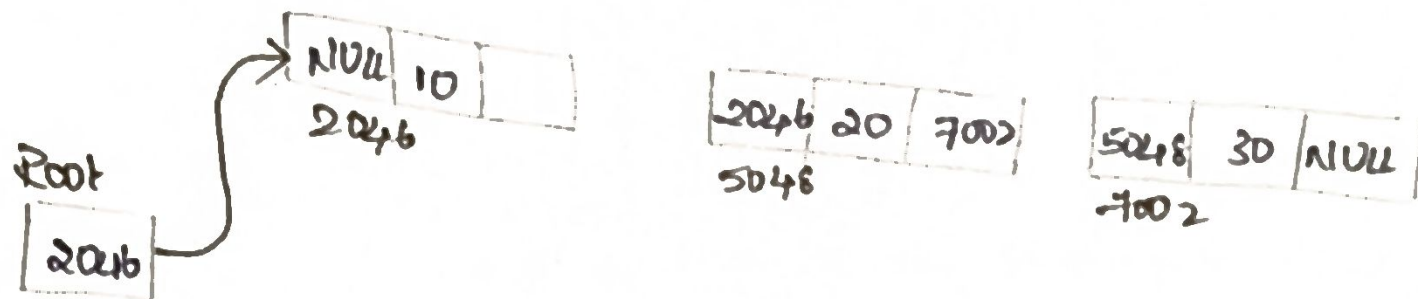
}

Algorithm:-

1. Start from the head node

2. If head == NULL, Print "List is Empty"

3. Otherwise, move through each node
4. Print the data of each node
5. Stop when temp == NULL



Traversal:-

- temp → 10 (Print 10)
- temp → 20 (Print 20)
- temp → 30 (Print 30)
- temp → Null (Stop)